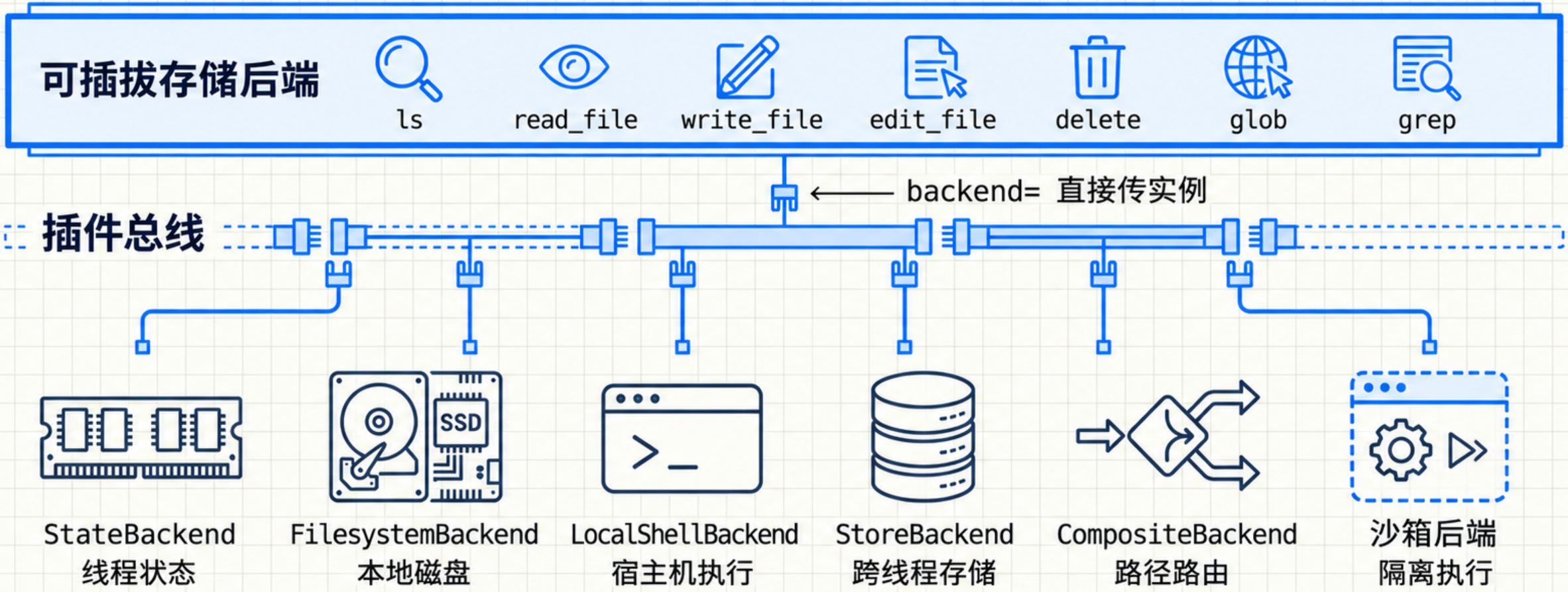
Deep Agents 实战

第3章·下篇：可插拔存储后端



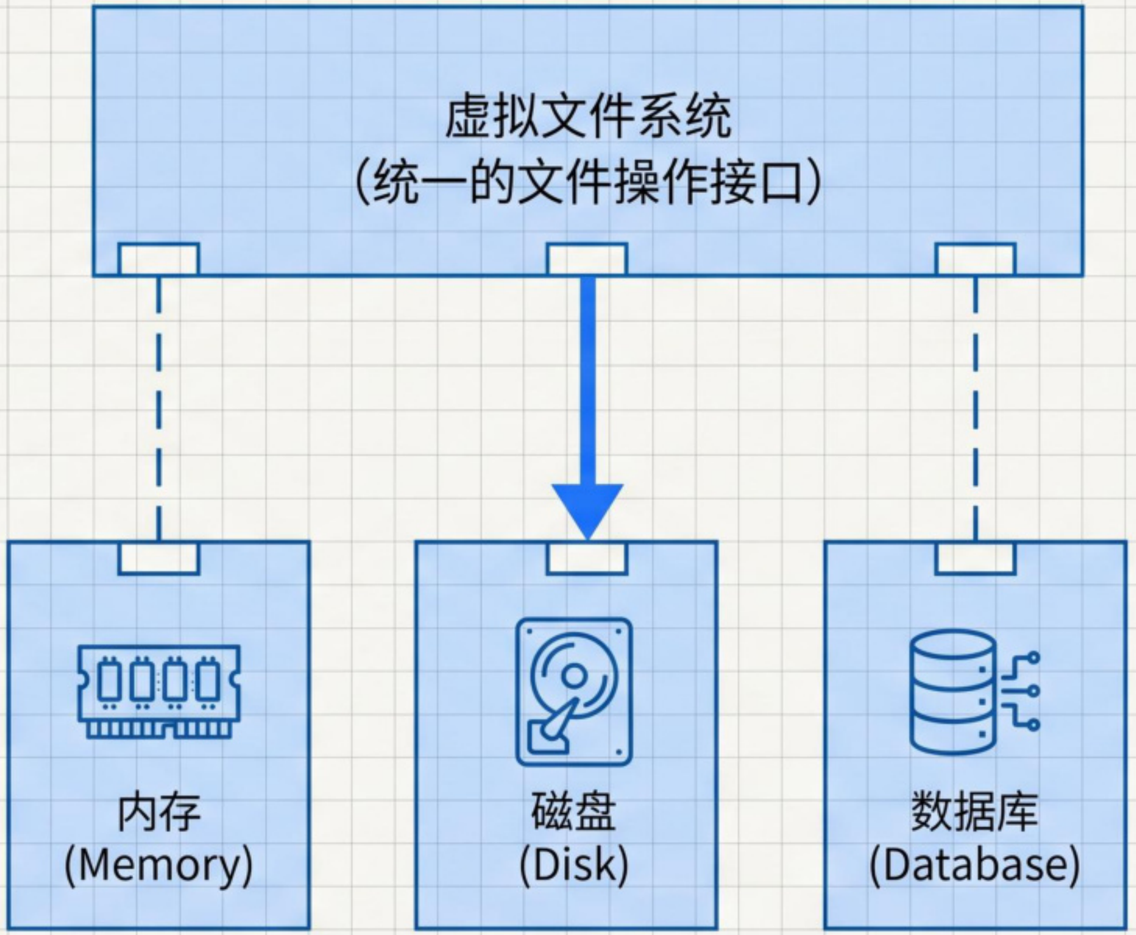
什么是存储后端？

虚拟文件系统背后的可插拔存储层

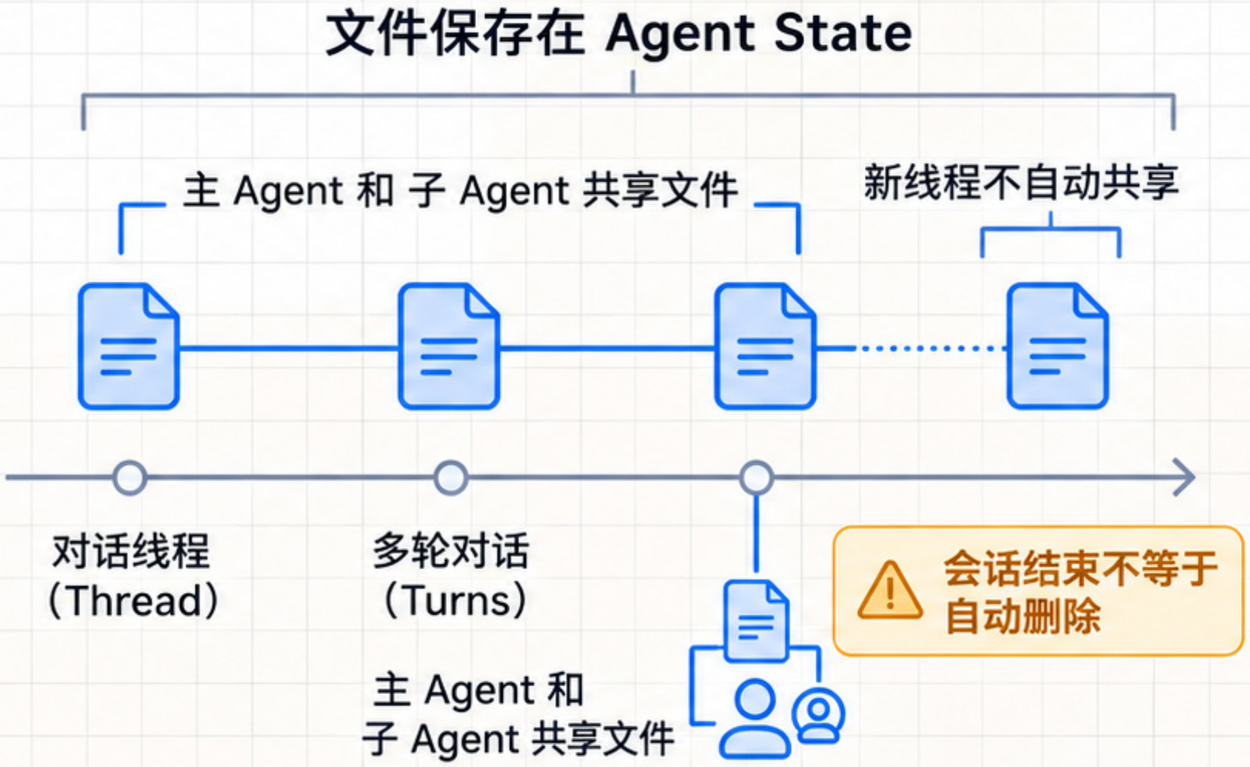
虚拟文件系统 = 统一的文件操作接口（ls、read、write...）

后端 = 文件实际存到哪里（内存？磁盘？数据库？）

可插拔设计：同一套工具，切换后端即切换存储策略



StateBackend：线程内的工作空间




```python
from deepagents import create_deep_agent
from deepagents.backends import StateBackend

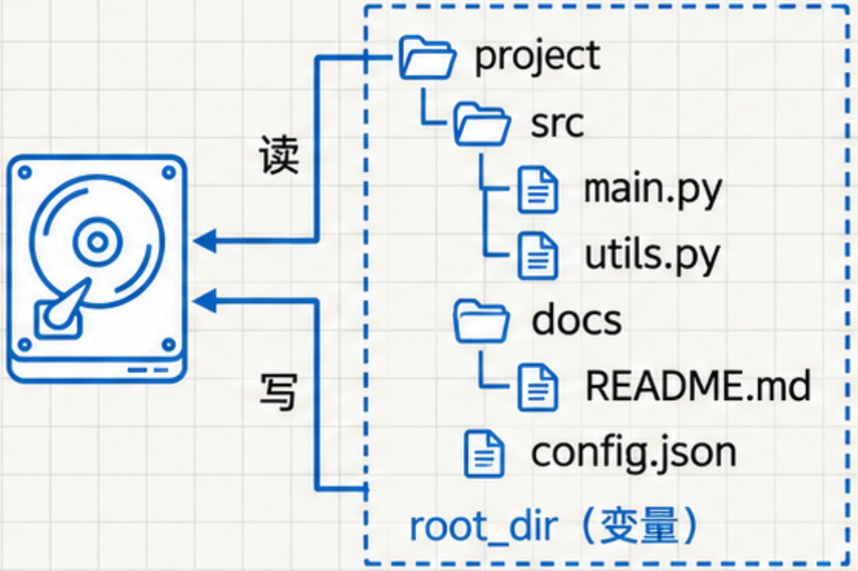
agent = create_deep_agent(
    model=model,
    backend=StateBackend(),
)
```

跨调用延续：同一 thread + checkpointer。

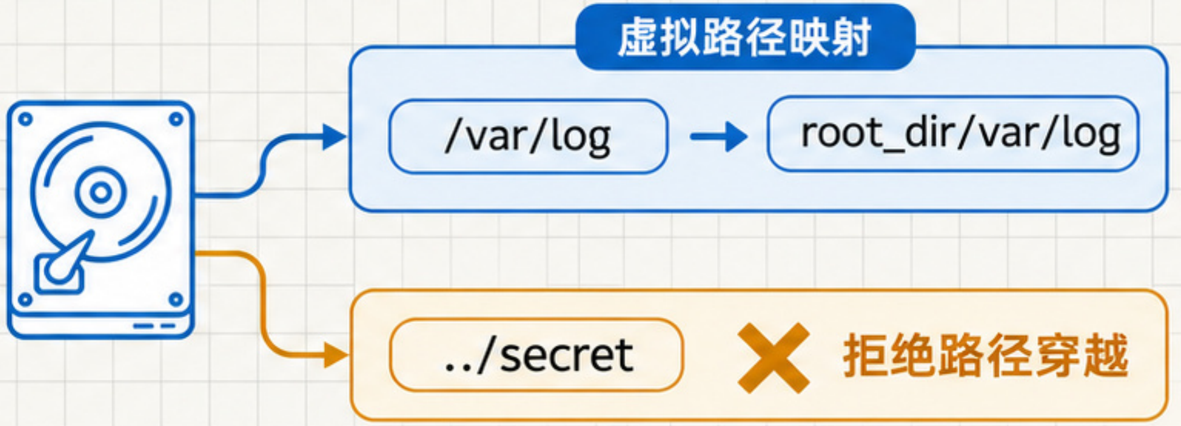
跨进程保留取决于持久化 checkpointer；上例仅展示后端配置。

FilesystemBackend：真实磁盘读写

root_dir 文件树（读写真实磁盘）



文件工具的路径范围（virtual_mode 机制）



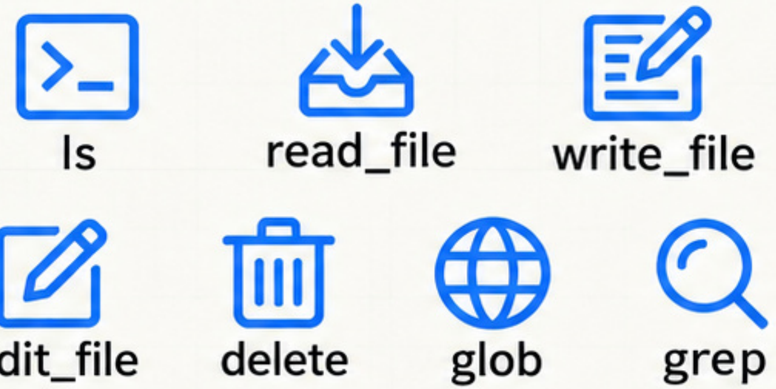
- root_dir 可为相对路径，初始化时解析为绝对路径。
- virtual_mode=True 限制文件工具的路径范围。
- virtual_mode=False 时，root_dir 不是访问边界。

```
FilesystemBackend(root_dir=".", virtual_mode=True)
```

⚠ 修改会影响真实文件，请先备份；不适合不可信多用户场景。

LocalShellBackend：不是执行沙箱

基础文件工具 (7)



execute on host terminal

Shell 命令直接运行在宿主机

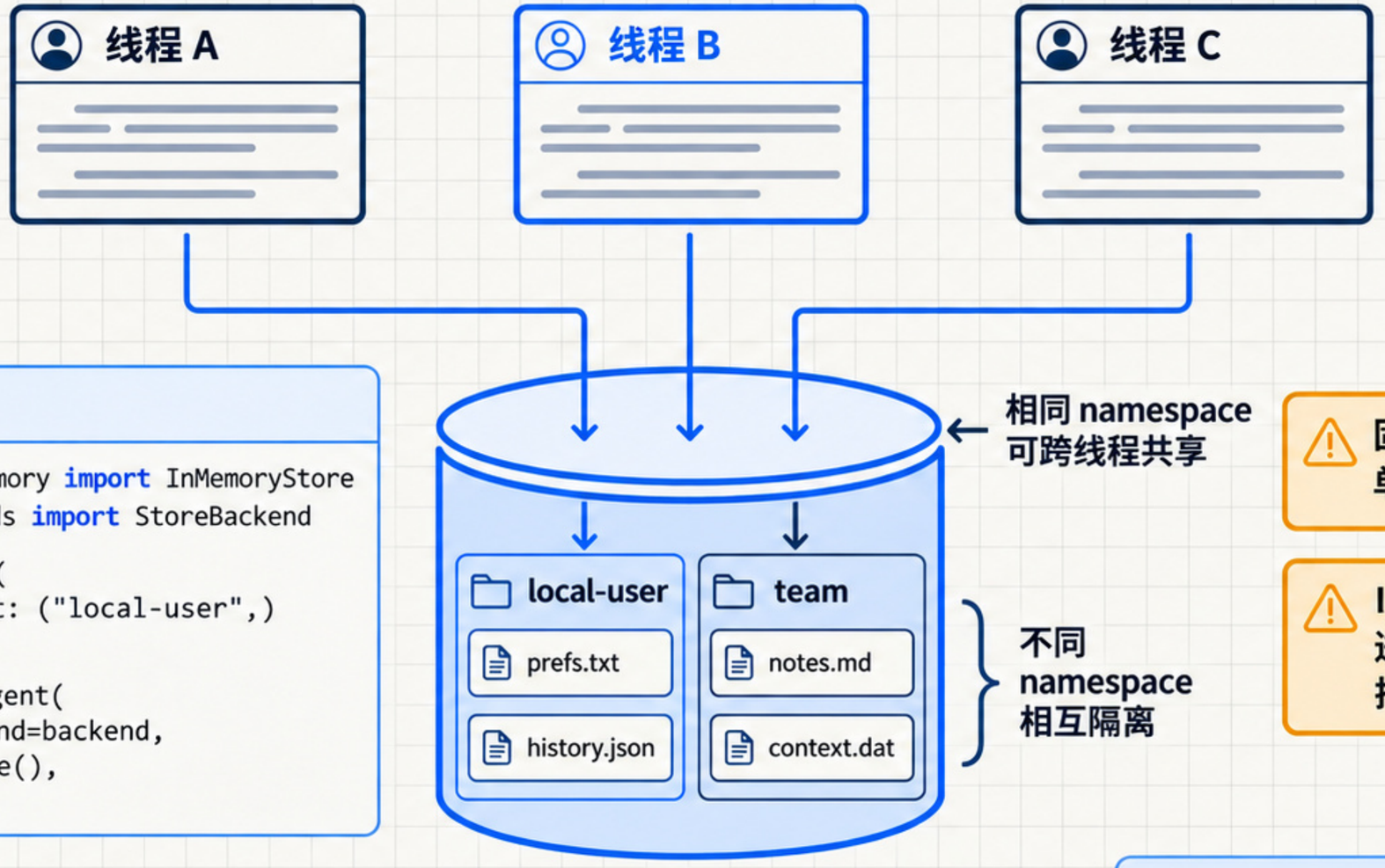
```
LocalShellBackend(
    root_dir=".",
    virtual_mode=True,
    env={"PATH": "/usr/bin:/bin"},
)
```

⚠ virtual_mode 只约束文件工具，不隔离 Shell。

root_dir 是工作目录，不是命令访问边界。

仅限可信本地开发；不可信代码应选隔离沙箱。

StoreBackend：跨线程共享文件



代码示例

```python
from langgraph.store.memory import InMemoryStore
from deepagents.backends import StoreBackend

backend = StoreBackend(
    namespace=lambda rt: ("local-user",)
)

agent = create_deep_agent(
    model=model, backend=backend,
    store=InMemoryStore(),
)
```

⚠ v0.7：传后端实例，不再传 backend 工厂函数。

⚠ 固定 namespace 仅用于单用户本地演示。

⚠ InMemoryStore 不保证进程重启后保留；生产需持久化 Store。

namespace 是必填参数。

namespace：先明确共享边界

1 按用户

用户偏好与私有记忆

→ 同一用户的线程共享

2 按线程

独立工作空间

→ 不同线程分离

3 按助手

共享知识库

→ 同一助手用户可能共享

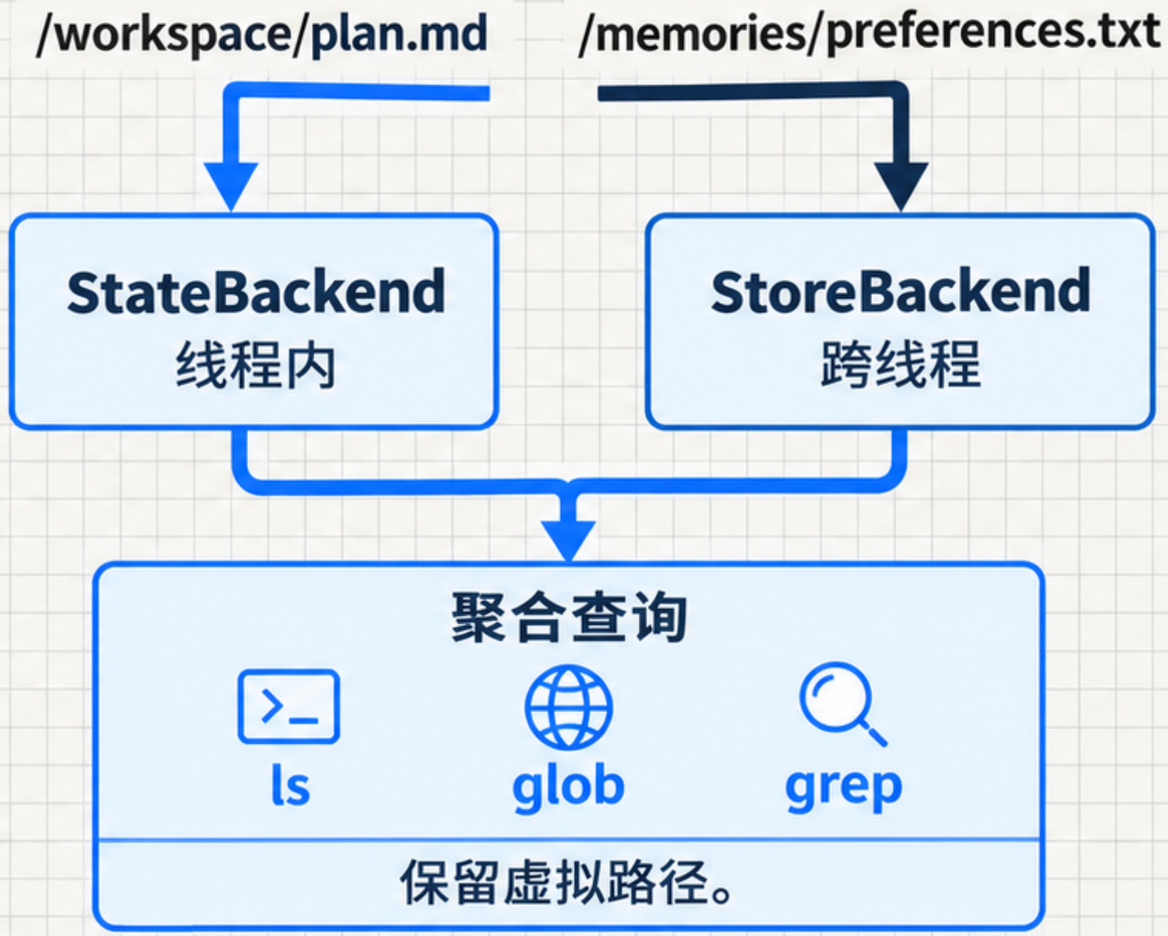
```python
def user_namespace(rt):
    if rt.server_info:
        return (rt.server_info.user.identity,)
    return ("local-user",)

backend = StoreBackend(namespace=user_namespace)
```

server_info 依赖运行环境；本地回退仅适合单用户演示。生产必须使用可信身份，不应让多用户共用固定回退值。

CompositeBackend：按路径路由

```
backend = CompositeBackend(
  default=StateBackend(),
  routes={
    "/memories/": StoreBackend(
      namespace=user_namespace,
    ),
  },
)
```



user_namespace 见前页；create_deep_agent 仍需传 store。

⚠ InMemoryStore 仅用于开发演示。

后端配置速查

以下仅展示配置片段；model、导入与身份函数需事先准备。

① StateBackend（状态后端）

```
StateBackend()
```

② FilesystemBackend（文件系统后端）

```
FilesystemBackend(root_dir=".",
                  virtual_mode=True)
```

③ LocalShellBackend（本地 Shell 后端）

⚠ 宿主机执行，无 Shell 隔离

```
LocalShellBackend(root_dir=".",
                  virtual_mode=True)
```

④ StoreBackend（持久化后端）

⚠ 同时传入 store；身份函数见前页

```
StoreBackend(namespace=user_namespace)
```

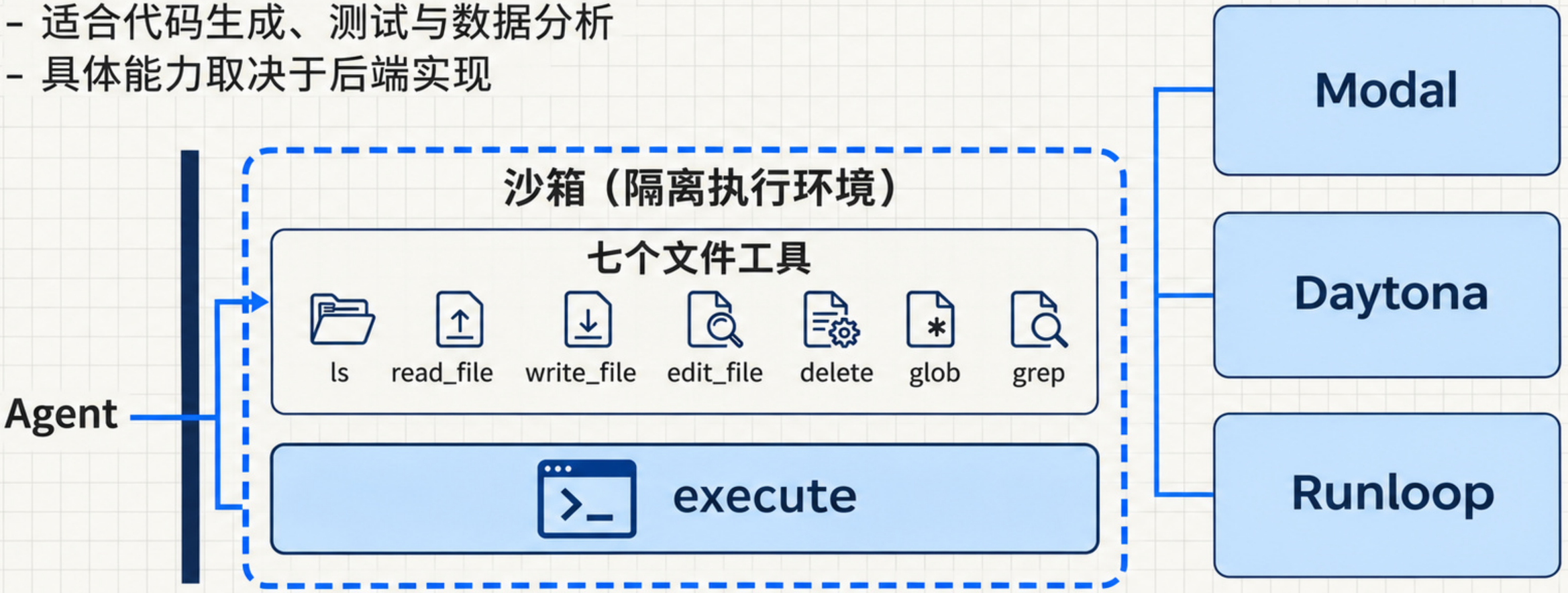
⑤ CompositeBackend（混合路由后端）

```
CompositeBackend(default=StateBackend(),
    routes={"/memories/": StoreBackend(
                namespace=user_namespace)})
```

⚠ root_dir="." 是相对路径；持久化与隔离需要分别配置。

沙箱后端：隔离执行环境

- execute 在隔离环境中运行命令
- 适合代码生成、测试与数据分析
- 具体能力取决于后端实现



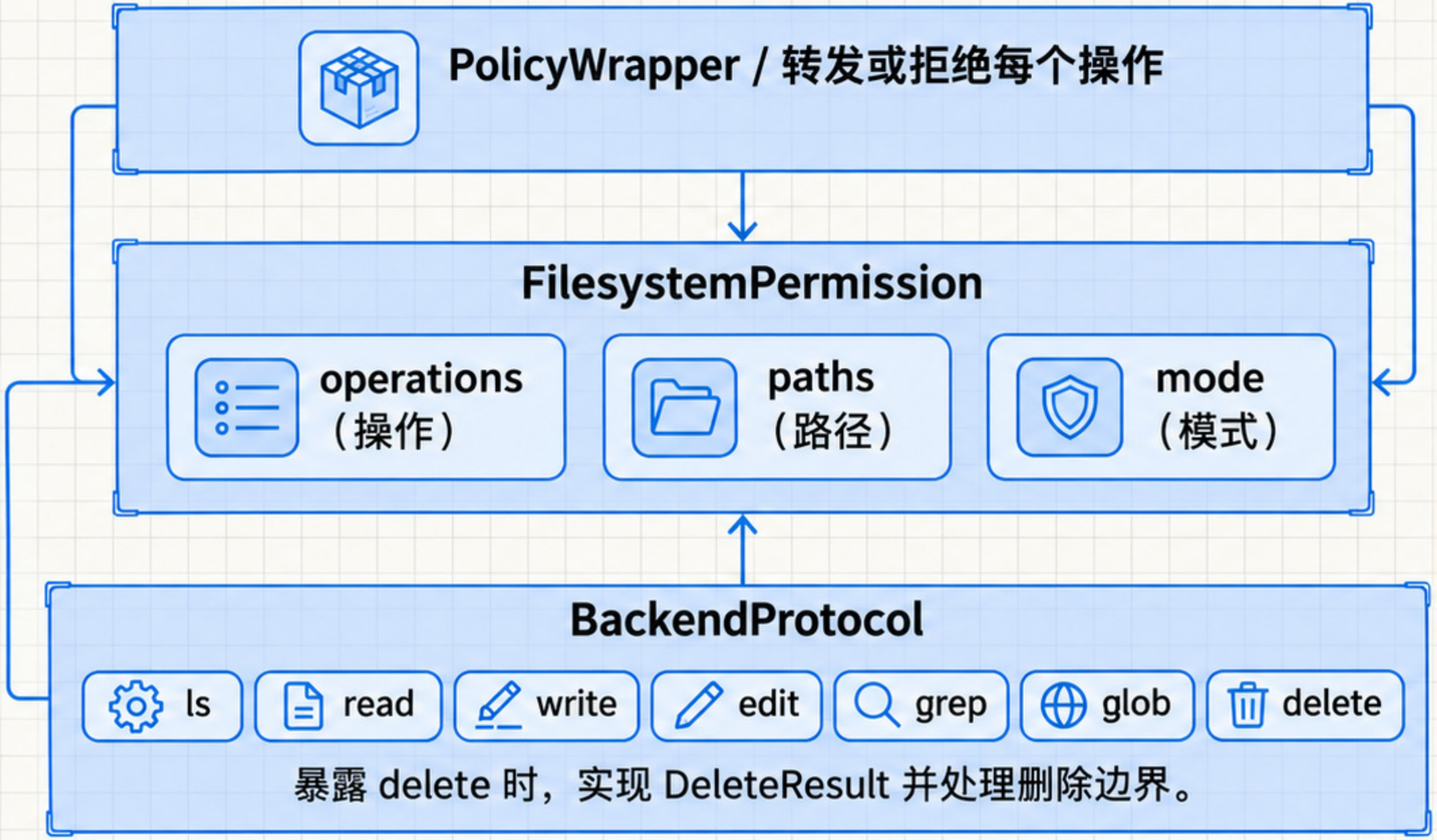
⚠ 隔离不等于绝对安全：仍需限制网络、凭据、挂载与资源权限。

后端选择：存储与执行分开考虑

	场景	推荐	注意事项
	学习实验	StateBackend	线程状态；延续依赖 checkpointer
	可信本地文件处理	FilesystemBackend	virtual_mode=True；操作真实文件
	可信本地命令执行	LocalShellBackend	不提供 Shell 隔离
	跨线程记忆	StoreBackend / Composite	namespace + 持久化 Store
	不可信代码执行	沙箱后端	审查网络、凭据与挂载权限
	混合工作空间	CompositeBackend	按路径组合存储策略

 **InMemoryStore 是开发方案，不等于持久化数据库。**

自定义后端与权限规则

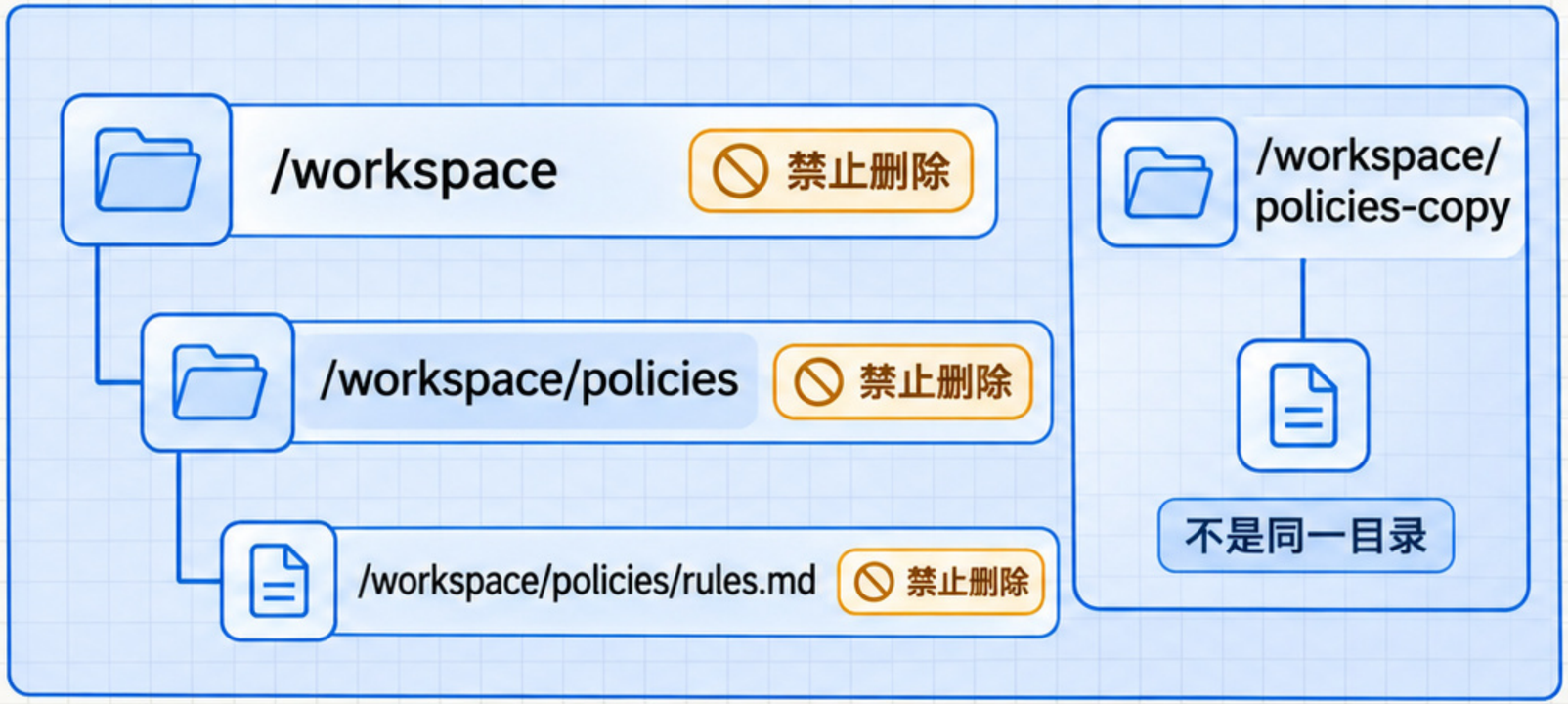


- 规则按顺序匹配，首个命中生效
- 未命中默认允许；具体规则放前面
- mode: allow / deny / interrupt
- interrupt 需要 checkpointer 与恢复流程

⚠ 删除属于 write 权限类别；
权限规则不等于 Shell 沙箱。

旧名 ls_info / grep_raw / glob_info 已弃用。

删除防护：目录边界不能漏



- 拒绝删除受保护目录本身
- 拒绝删除其子孙路径
- 拒绝删除包含它的祖先目录
- 按路径边界判断，不能只用字符串前缀

⚠ 拒绝 .. 路径穿越；符号链接与别名需由后端约束。

⚠ 只关闭 delete 工具，不能阻止 Shell 或自定义工具删除。

本篇回顾



1. 同一套文件工具，后端决定存储与执行方式。



2. State 按线程；Store 按 namespace；持久化需相应存储。



3. v0.7 直接传后端实例，namespace 回调仍可使用。



4. virtual_mode 约束文件路径，不是 Shell 沙箱。



5. 自定义后端必须覆盖 delete 与目录删除边界。



6. 权限首个匹配生效；未匹配默认允许。

下一章：任务规划与中间件。